IMAP

Introduction to Modern App Development

Unit 1 Notes: Front End Development

Contents

1. Course Orientation

Unit 1, Front End Development, builds the base for the rest of the course. Everything downstream, Django on the back end, REST APIs, React on the front end, sits on top of the ideas covered here: how a browser talks to a server, how a page is structured, styled and made interactive, and how a team tracks changes to code with Git.

The unit has five threads that fit together into one picture of how a modern web app is built and shipped:

- The client server model: how a browser and a server exchange requests and responses.
- HTML: the structure and meaning of a page.
- CSS: how that structure is styled and laid out.
- JavaScript: how the page behaves and reacts, including talking to a server asynchronously.
- Git and GitHub: how the code for all of the above is saved, shared and merged as a team.

2. The Client Server Model

A web application is really a conversation between two computers: a client, usually a browser, and a server, a machine that stores the application and its data. Neither can do much alone. The client asks; the server answers.

2.1 What happens when you hit a URL

In the simplest case, a browser sends a request to a server and the server sends back a response. In practice, most traffic on the real internet does not travel directly from client to server. It is rerouted through the internet, that is, through DNS resolution and a chain of networking infrastructure, before it reaches the correct server, and the response is routed back the same way. Conceptually this is still just request out, response back, only with the internet sitting in the middle as the delivery system.

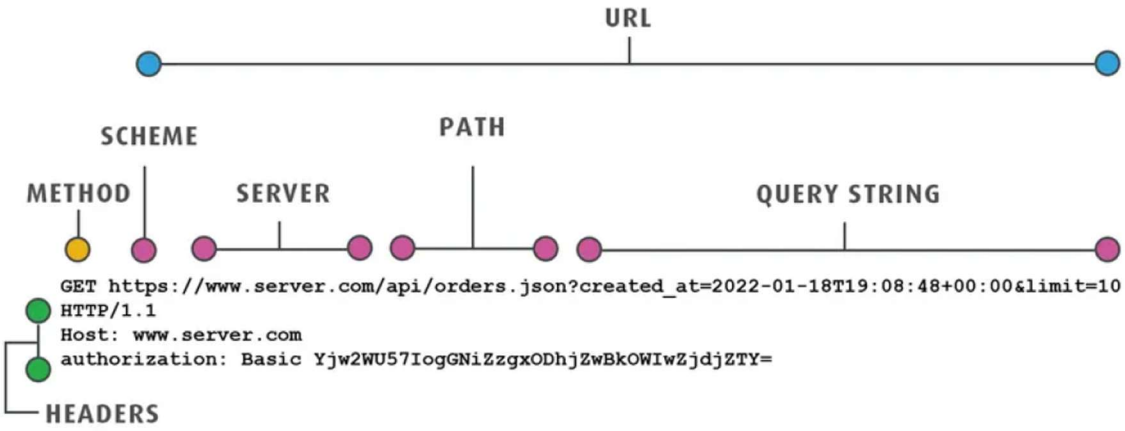
2.2 Anatomy of a request

Every HTTP request is built from a small, fixed set of parts, and recognising them is what makes reading an API call or a browser network tab straightforward:

- **Method:** The action being requested, for example GET to read data or POST to create data.
- **Scheme:** The protocol, almost always https in a production application.
- **Server:** The host being contacted, for example www.server.com.
- **Path:** Which resource on that server is being asked for, for example /api/orders.json.

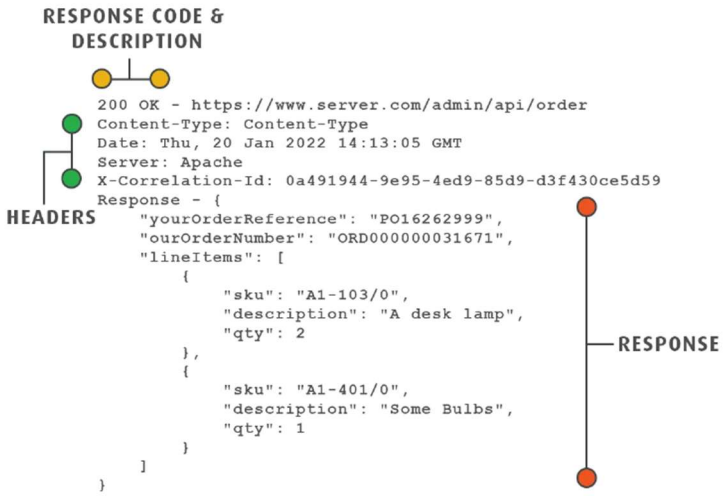
- **Query string:** Extra key value pairs appended after a question mark, used for filtering, pagination or sorting, for example ?created_at=...&limit=10.
- **Headers:** Metadata about the request that sits outside the URL, such as which host to contact and an authorization token.

> **Note:** The URL alone only carries the method, scheme, server, path and query string. Authorization and other metadata travel separately as headers, which is why two requests to the exact same URL can behave completely differently depending on who is asking.



2.3 Anatomy of a response

A response mirrors a request: it also carries headers and a body, plus one piece a request does not have, a status code and description, for example 200 OK, that tells the client in one glance whether the request succeeded, failed, or needs to be redirected. The body of a response is very often JSON: a nested structure of keys and values that both the browser and server understand, which is why JSON has become the default data format for APIs.



3. How a Browser Renders a Webpage

Once the response for an HTML page arrives, the browser still has to turn a text file into pixels on a screen. That happens in a fixed pipeline, and understanding it explains a surprising number of practical rules, such as why stylesheets belong in the head and scripts belong at the end of the body.

- **1. Fetching:** The browser sends the request and the server responds with the HTML, CSS and other files the page needs.
- **2. Parsing HTML:** The browser reads the HTML top to bottom and builds the DOM, the Document Object Model, a tree of objects that mirrors the nesting of the tags.
- **3. Processing CSS:** The browser parses every stylesheet and builds the CSSOM, the CSS Object Model, a second tree that records which styles apply to which elements.
- **4. Constructing the render tree:** The DOM and CSSOM are combined into a render tree, which keeps only the elements that will actually be visible and attaches their computed styles.
- **5. Layout and painting:** The browser calculates the exact position and size of every box, this step is called layout, and then draws the actual pixels to the screen, this step is called painting.

> **Note:** This pipeline is the reason CSS is called render blocking: the browser will not paint anything until the CSSOM is ready, so a slow stylesheet delays the whole page. It is also the reason changing layout affecting properties such as width or top is expensive, it forces layout to run again, while changing colour only forces a repaint. This distinction matters directly for writing efficient CSS animations, covered in section 5.

4. HTML: Structure and Meaning

> **Lab manual reference:** *The complete list of HTML tags and attributes, grouped by document structure, text, links, lists, media, tables, forms and semantic layout, is in the Lab 1 Manual. These notes cover only the ideas behind that reference.*

4.1 Anatomy of a basic HTML page

A minimal HTML page has four nested parts that map directly onto what the user sees:

- **<!DOCTYPE html>** A one line declaration that tells the browser to render the page using the modern HTML5 rules.
- **<html>** The single root element that wraps the entire page, everything else lives inside it.
- **<head>** Invisible information for the browser and search engines: the character set, the viewport setting and the page title.
- **<body>** Everything the user actually sees: headings, paragraphs, images and so on.

4.2 Semantic versus non semantic HTML

A semantic tag describes what its content means, not just how it should look. Tags such as header, nav, main, section and footer tell the browser, screen readers and search engines the role each part of the page plays. Non semantic tags, div and span, are generic containers with no built in meaning: they exist purely for grouping and styling.

Semantic HTML is preferred wherever a matching tag exists because it improves accessibility, since screen readers use the semantics to describe the page to a user who cannot see it, and it improves SEO, since search engines use the same structure to understand what a page is about. A div is still the right choice whenever a wrapper has no semantic role of its own, for example a purely decorative styling wrapper.

Feature	Semantic HTML	Non semantic HTML
Accessibility	Clear, screen readers understand structure	Varies, depends entirely on added ARIA attributes
SEO	High, search engines read structure directly	Low, no inherent meaning to read
Clarity for developers	Easy, the tag name explains the role	Requires class names to explain intent

5. CSS: Presentation and Layout

> **Lab manual reference:** *The full property by property reference for selectors, the box model, units, colour, typography, backgrounds, display, positioning, Flexbox, Grid, custom properties, animation and responsive design is in the Lab 2 Manual. These notes cover the reasoning, the cascade, and the decisions behind that reference.*

5.1 Where CSS fits in the pipeline

As covered in section 3, CSS controls two stages of rendering: the browser builds the CSSOM while parsing stylesheets, and it uses that CSSOM during layout and paint. Because CSS is render blocking, stylesheets are linked in the head so the CSSOM is ready before anything is painted, while scripts are placed at the end of the body so they do not delay the first paint of the page.

5.2 Three ways to add CSS, and why external wins

- **Inline:** A style attribute directly on one element. Cannot be reused and is very hard to override later, avoid it.
- **Internal:** A style block inside the head of a single page. Fine for a one off demo, but cannot be shared across pages.
- **External:** A separate .css file linked with a link tag. The browser can cache it and every page on the site can share it, this is the default choice for a real project.

5.3 The cascade: how conflicting rules are resolved

The cascade is the C in CSS, and it is the algorithm the browser runs whenever two rules target the same element and disagree about a property. It resolves the conflict in a strict order, and each step only matters if the previous step ended in a tie:

- **1. Importance:** A declaration marked !important wins outright over anything not marked !important, regardless of specificity or source order.

- **2. Specificity:** Among declarations of equal importance, the more specific selector wins. Specificity is scored as three numbers, IDs, classes and attributes, then elements, compared left to right, so a single class always beats any number of element selectors, and a single ID always beats any number of classes.
- **3. Source order:** If importance and specificity are tied, the declaration that appears later in the stylesheet, or in a later linked stylesheet, wins.

> **Note:** The cascade resolves one property at a time, not one whole rule at a time. A single element can legitimately take its colour from one rule and its margin from a completely different rule. !important should be a last resort, since it skips specificity entirely and the only way to override it later is with another, even more forceful, !important.

5.4 Inheritance

Separately from the cascade, some CSS properties are inherited automatically from parent to child, mainly text related properties such as colour, font family and line height. Most other properties, including margin, padding, border, width and height, are not inherited and reset to their default on every new element. This is why setting a font on the body affects the whole page, but setting a margin on the body does not push every element on the page outward.

5.5 The box model and box-sizing

Every element on a page is really four rectangles nested inside one another, from the inside out: content, padding, border and margin. Padding sits inside the border and takes on the element's background colour; margin sits outside the border and is always transparent, its job is to push neighbouring elements away.

The box-sizing property decides what the declared width actually measures. Under the default, content-box, padding and border are added on top of the declared width, so a 300px wide box with 10px of padding and a 5px border actually occupies 330px on screen. Under border-box, padding and border are absorbed inside the declared width, so that same box stays exactly 300px wide. Because border-box is the far easier mental model for layout work, it is applied globally at the top of almost every real stylesheet:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

> **Note:** Margin collapse: two vertical margins that touch do not add together, the larger one wins and the smaller one simply disappears. Only vertical margins collapse, and only outside of flex or grid containers, which is one reason gap is now generally preferred over margin for spacing items inside a flex or grid layout.

5.6 Choosing units

px is a fixed size that ignores the user's own font size preference, so it is best kept for details that genuinely should not scale, such as thin borders. rem is relative to the root font size and rem based font sizes and spacing grow correctly if a user increases their browser's default text size, which is why rem is the recommended default for font size and spacing. em is relative to the current element's own font size, which means it multiplies awkwardly when nested and is generally better replaced with rem. Percentages and the viewport units vw and vh are useful for fluid widths and full screen sections, while calc(), clamp() and min()/max() let a single value mix units or stay within a minimum and maximum bound without a media query.

5.7 Display and positioning

The display property decides how a box behaves in the flow of the page: block elements start on a new line and take the full available width, inline elements sit inside a line of text and ignore width and height, and inline-block elements sit inside a line of text but still respect width and height. display: none removes an element from the page entirely, taking up no space, which is different from visibility: hidden, which hides the element but still reserves its space in the layout.

The position property, by contrast, decides whether an element follows the normal flow at all. static is the default and ignores top, left and z-index completely. relative keeps an element's space in the flow but lets it be nudged visually. absolute removes the element from flow entirely and positions it against its nearest positioned ancestor, so an absolutely positioned badge is usually paired with a parent set to position: relative, otherwise it jumps to the corner of the whole page. fixed positions against the browser window itself and ignores scrolling, and sticky behaves like relative until a scroll threshold is reached, then behaves like fixed. z-index only has any effect on positioned elements, that is, anything except static.

5.8 Flexbox: one axis at a time

Flexbox lays out items along a single line, either a row or a column, and every Flexbox property is really named after one of two axes: the main axis, the direction items are laid out in, and the cross axis, perpendicular to it. justify-content always controls spacing along the main axis and align-items always controls alignment along the cross axis. Setting flex-direction: column swaps which physical direction each axis points in, which is the single most common source of Flexbox confusion, since justify-content then moves items up and down instead of left and right.

A useful shortcut worth remembering by heart is that display: flex; justify-content: center; align-items: center centres any element, both horizontally and vertically, at any size, which is why it has effectively replaced older centring hacks.

5.9 Grid versus Flexbox

Grid and Flexbox are not competitors, they solve different shaped problems and most real pages use both together. Flexbox is one dimensional: it lays out a single row or a single column and lets the items size themselves based on their content. Grid is two dimensional: rows and columns are defined together up front, and the layout is driven by those tracks rather than by the content. In practice, Grid is generally used for the page skeleton, the overall header, sidebar and content arrangement, while Flexbox is used inside individual components, such as a navbar or a row of cards.

5.10 Custom properties, transitions and animation

A custom property, written as --name: value and read back with var(--name), stores a value once, typically on the :root selector, so it can be reused everywhere and updated from a single place. This is the mechanism behind theming: a dark mode toggle can be implemented as one media query or one class that overrides a handful of variables, with no JavaScript required to restyle the rest of the page, since every component already reads its colours through var().

For motion, a transition smooths a single property between two states, typically triggered by a hover or a class toggle, while a @keyframes animation defines and plays a full, named sequence of steps on its own. Both should be built almost entirely out of the transform and opacity properties, because these two are the only properties a browser can animate without recalculating layout for the whole page, which is what keeps an animation smooth. Animating width, height, top or left forces a layout recalculation on every single frame and tends to look noticeably jerky.

> **Note:** Motion should also respect users who have asked for less of it: wrap non essential animation in @media (prefers-reduced-motion: reduce) so it can be turned off system wide.

5.11 Mobile first responsive design

Mobile first means writing the base styles for the smallest, most constrained screen first, and then layering on complexity for larger screens using min-width media queries, rather than starting from a desktop layout and stripping it down with max-width queries. The mobile first order means every rule only ever adds capability, nothing has to be undone at a smaller breakpoint, and it also forces an honest decision about what content actually matters most, since the narrowest layout has to work first. Breakpoints are best chosen by widening the browser until a specific layout actually looks broken, rather than by memorising exact device widths, since device sizes change every year.

6. CSS Frameworks: Bootstrap and Tailwind

A CSS framework is simply pre written, pre tested CSS that a project pulls in instead of hand styling everything from scratch. In exchange for some control over fine visual detail, a framework buys consistent, cross browser tested styling and a much faster path from an empty page to a working layout.

Bootstrap, released in 2011, is component based: a single class such as btn btn-primary names an entire finished look, and the developer thinks in terms of ready made pieces, cards, navbars, modals. Tailwind, released in 2017, is utility first: there is no ready made button class, instead many small classes are stacked together in the HTML, one class per CSS property, and the developer effectively builds the design by hand, one property at a time, without leaving the markup.

	Bootstrap	Tailwind
Approach	Ready made components	Small, single property utility classes
Developer thinks in	Finished pieces, e.g. a card	Individual CSS properties, e.g. padding, colour
Custom look	Harder, sites can look similar	Easy, the design is composed by hand
Best when	Speed and sensible defaults matter most	A unique, tightly controlled design matters most

> **Note:** Neither framework is objectively better, they are a trade off between speed of a sensible default look and control over a unique one. Many production teams use Tailwind today for exactly the control it offers, but Bootstrap remains a fast, reasonable default when consistency matters more than originality.

7. JavaScript: Behaviour

> **Lab manual reference:** *The exact syntax for array methods, DOM selection and manipulation, event handling and Promise based APIs is in the Lab 3 Manual. These notes focus on the underlying rules, especially scoping, typing, equality and the evolution from callbacks to async/await, which the manual assumes as background.*

If HTML is the structure of a page and CSS is its presentation, JavaScript is its behaviour: it responds to clicks, typing and scrolling, changes the page after it has already loaded, fetches data from a server without a refresh, and generally makes a static document into an application. All of it runs inside the browser, and the DevTools console, opened with F12, is the fastest place to try a line of JavaScript directly.

7.1 Variables and block scope

const is the default choice: the variable name cannot be pointed at a new value after it is declared. This is not the same as the value being frozen, an array or object stored in a const can still have its own contents changed, the lock is only on the variable name itself. let is used only when a value genuinely needs to be reassigned later, such as a running counter. var, the original way to declare a variable, should not be used in new code, because it is function scoped rather than block scoped: a variable declared with var inside an if block or a loop leaks out of that block, which historically caused a large share of JavaScript bugs. const and let both respect block scope, they exist only inside the { } they are declared in, which is the main reason var was retired.

7.2 Data types

JavaScript values fall into two families. Primitive types, string, number, boolean, null, undefined and symbol, are copied by value, so two variables holding a primitive are always independent of each other. Reference types, arrays, objects and functions, are copied by reference, so two variables can point at the exact same array in memory, and changing it through one variable is visible through the other. A variable itself has no fixed type in JavaScript, the value it currently holds carries its own type, which is why typeof can return a different answer for the same variable at different points in a program. One long standing quirk worth remembering is that typeof null returns "object", for historical reasons rather than logical ones.

Primitives	Reference types
Copied by value	Copied by reference
string, number, boolean, null, undefined, symbol	array, object, function
Two variables are always independent	Two variables can point at the same underlying data

7.3 Equality and truthy values

=== compares both value and type and never converts anything, while == quietly converts types before comparing, which is why 5 == "5" evaluates to true and can hide real bugs. === and !== should be used everywhere. Separately, any value used inside an if condition is coerced to true or false. Only a small, fixed set of values are falsy: false, 0, an empty string, null, undefined and NaN. Every other value is truthy, including some values that look empty at first glance, such as the string "0" and an empty array or object.

7.4 Functions and arrow functions

A function is a reusable block that takes named inputs, its parameters, and can send a value back with return; if a function has no return statement, calling it evaluates to undefined. Arrow functions, introduced in ES6, are a shorter syntax for the same idea: a single expression body needs no braces and no explicit return, which is why arrow functions read especially cleanly inside array method callbacks such as nums.map(n => n * 2). Arrow functions also behave differently from ordinary functions with respect to the keyword this: an arrow function does not create its own this, it keeps whatever this was in the surrounding code, which matters particularly inside object methods and event handlers.

7.5 ES6+ essentials

- **Template literals:** Backtick strings that embed expressions directly with ${ }, replacing string concatenation with +.
- **Destructuring:** Unpacking values out of an object or array in a single line, for example const { name, age } = user.
- **Spread and rest:** Three dots that either expand a value, for example combining two arrays with [...a, ...b], or gather multiple arguments into one array.
- **Optional chaining:** user?.address?.city safely reads a nested property and evaluates to undefined instead of throwing if something along the chain is missing.

7.6 Array methods, the mindset

map, filter and reduce are the backbone of working with lists of data, and they share one important property: each one returns a brand new array or value and leaves the original array completely untouched, which is exactly what makes them safe to chain together, one after another, into a single readable pipeline. A chain such as nums.filter(n => n > 0).map(n => n * 2) reads naturally left to right as a sentence: keep the positive numbers, then double each one. Chains like this now replace most of what used to be written as explicit for loops.

7.7 Selecting and changing the DOM

State should generally live in an ordinary JavaScript variable, and the DOM should simply be updated to reflect it, rather than being treated as the source of truth itself. A typical interaction follows the same four step loop every time: select the relevant element, listen for an event on it, update some piece of state in memory, then write that updated state back into the DOM. This update state, then render pattern is exactly the mental model that frameworks such as React, covered later in the course, formalise and automate.

> **Note:** Prefer .textContent and .classList over .innerHTML wherever possible. .innerHTML parses and inserts raw HTML, which becomes a security risk the moment any part of that HTML came from user input, since it can inject a working script tag into the page.

7.8 Event delegation

Rather than attaching one listener to every individual child element, a single listener can be attached to their shared parent, using event.target inside the handler to work out which specific child actually triggered the event, often combined with el.closest(selector) to find the nearest matching ancestor. This pattern also

automatically covers child elements that are added to the page later, after the listener was attached, since the listener lives on the parent and never needs to be reattached.

7.9 The asynchronous problem

JavaScript runs on a single thread, meaning it can only do one thing at a time. A slow operation such as a network request would freeze the entire page if the browser simply waited for it to finish before doing anything else. The language's approach to asynchronous work has evolved through three stages, each one built to fix a problem with the one before it:

- **Callbacks:** A function passed in to be run once an operation finishes. Nesting several of these to run one after another becomes deeply indented and hard to follow, often called callback hell.
- **Promises:** An object representing a value that is not ready yet. It starts pending and settles into either fulfilled, with a value, or rejected, with an error, and .then()/.catch() chain cleanly onto it.
- **Async and await:** Syntax written directly on top of Promises that lets asynchronous code be written and read top to bottom, like ordinary synchronous code, while still not freezing the page.

async/await is not a new mechanism, it is nicer syntax layered directly over Promises, and the same try/catch used for ordinary thrown errors catches a rejected await, which keeps error handling consistent across synchronous and asynchronous code.

7.10 Sequential versus parallel awaiting

Two awaits written one after another run sequentially: the second request does not even start until the first has fully finished, so the total time taken is the sum of both. When two requests do not actually depend on each other's results, they should instead be started together and waited on with Promise.all, so the total time taken is roughly the time of the slower of the two requests, not the sum of both. The most common version of this mistake is placing await inside a for loop over a list of independent requests, which quietly serialises all of them; the fix is almost always to build an array of the not yet awaited promises first and then await Promise.all(that array) once, outside the loop.

> **Note:** The other three common async mistakes worth checking for in any bug: forgetting await entirely, which leaves a pending Promise object in the variable instead of its value; leaving a rejected Promise with no catch or try/catch, which can fail silently; and awaiting something that is not actually a Promise, which is harmless but pointless. When a value is unexpectedly undefined or shows as a pending Promise in the console, a missing await is the first thing to check.

7.11 Modules

The export keyword marks which functions or values a file makes available to other files, and import pulls specific named pieces into another file, for example import { add } from './math.js'. Splitting a project into small, focused files instead of one large script is what modules are for, and a script tag that loads a module needs type="module" set on it.

8. Git and GitHub

Lab manual reference: *The exact commands for everyday Git use, branching, remotes and pull requests are listed in the Lab 3 Manual. These notes explain the model behind Git, why each command exists, and how to choose the right way to undo a mistake.*

8.1 Why version control exists

Without any version control, a project tends to end up as a folder full of files named things like app_final.js, app_final_v2.js and app_final_REAL.js, with no record of which one actually works or what changed between them. Version control replaces that folder of guesses with one file that keeps its entire history: every version is preserved without cluttering the working folder, every change carries a message explaining why it was made, the project can be returned to any earlier point in seconds, and a whole team can work on the same project in parallel without overwriting each other's work.

8.2 Git's mental model: snapshots, not differences

Git does not store a running list of line by line differences between versions. Each commit is a complete snapshot, a full photo, of the entire project at the exact moment that commit was made. Commits link together into a chain, and that chain is the project's history. Every commit carries an ID, a SHA hash, an author, a timestamp, a message and a pointer back to its own parent commit. HEAD is simply a label for whichever point in that chain is currently checked out.

8.3 Git versus GitHub

Git and GitHub are frequently confused but are not the same kind of thing. Git is a program, written by Linus Torvalds in 2005, that runs entirely on a local machine, works fully offline, and only tracks versions and history. GitHub is a website that hosts a shared copy of a Git repository online and adds collaboration features on top of Git itself, such as pull requests and issue tracking; it needs an internet connection and is only one of several such hosting services, GitLab being another. A useful way to hold the distinction: Git is the camera that takes the snapshots, GitHub is the shared album the snapshots get uploaded to.

8.4 The four places a change lives

A single change to a file physically passes through four distinct places before it is shared with a team, and each Git command moves it from one place to the next: the working directory, the files as currently being edited; the staging area, changes that have been picked out for the next commit; the local repository, commits that have already been saved on this machine; and the remote, the shared copy hosted on GitHub. git add moves a change from the working directory into staging, git commit moves staged changes into the local repository, and git push moves local commits up to the remote.

Note: The staging area is the step most people question at first. It exists so that a commit can represent one clean, logical change, rather than an unfiltered dump of every single edit sitting in the working directory at that moment. Running git commit before git add commits nothing at all, since nothing has been staged yet.

8.5 Branching

A branch is simply a movable pointer to a commit, not a copy of the whole project. Creating a branch and working on it keeps the main branch completely untouched and deployable while a feature is being built, and the branch is only folded back in, merged, once that work is finished and ready.

8.6 Merge conflicts

Git can merge two branches automatically whenever their changes touch different parts of a file. A conflict only happens when both branches have edited the exact same lines differently, and at that point Git genuinely cannot choose on its own, so it pauses and marks the disputed section directly inside the file, using <<<<<<<, ======= and >>>>>>> marker lines to show HEAD's version above the divider and the incoming branch's version below it. A conflict is not a sign that something is broken, it is Git asking a question. It is resolved by opening the file, deciding which lines to keep, deleting the marker lines themselves, then staging and committing the file as usual.

8.7 Undoing things: restore, reset and revert

Git offers three different commands for undoing work, and the right one depends entirely on how far that work has already travelled through the four places described above:

Command	Undoes	Use it for
git restore <file>	Throws away uncommitted edits, back to the last commit	A bad edit that was never committed
git reset	Moves HEAD backwards; --soft keeps the changes staged, --hard deletes them. Rewrites history.	A local commit that nobody else has seen yet
git revert <id>	Creates a brand new commit that undoes an old one. History is preserved, not rewritten.	A commit that has already been pushed and shared with a team

> **Note:** The golden rule underneath this table: never rewrite history that others have already pulled. Once a commit has been pushed and shared, it is undone with revert, which adds a new commit, never with reset, which erases the old one and can silently break every teammate's copy of the history.

8.8 Merge versus rebase

Merge and rebase are two different ways to bring a feature branch's commits into main, and they produce different shaped history. A merge combines both histories with a new merge commit, so both the original branch commits and a new commit tying them together remain visible in the history. A rebase instead rewrites history by replaying the feature branch's commits one by one on top of the current tip of main, producing a straight, linear history with no separate merge commit at all, but the replayed commits are technically new commits with new IDs. Because rebase rewrites commit history, it follows the exact same golden rule as reset: it should only be used on local commits that have not yet been pushed and shared.

8.9 .gitignore

A .gitignore file lists patterns for files and folders that Git should never track in the first place, typically dependency folders such as node_modules, environment files holding secrets such as .env, and build output. It only prevents new, currently untracked files from being picked up, once a file has already been committed, adding it to .gitignore afterwards does not remove it from the repository's history.

8.10 The pull request workflow

A pull request, PR, proposes merging one branch into another and gives a team a dedicated place to review a change before it lands in main. The standard flow is to branch off main, make a small, focused, honestly described change, push that branch, not main, since most contributors will not have permission to push directly to someone else's main, and then open a pull request comparing the branch against main so the repository owner can review and merge it.

Good commit habits make this whole process easier for a reviewer to follow: keep each commit small and focused on one idea, write commit messages in the present tense, for example Add login form rather than added stuff, pull the latest changes before starting new work and push when pausing, and let the commit message explain why a change was made, since the diff itself already shows what changed.